Monadic Types and the `foreach` Comprehension in f1r3|@ng

A synchronous comprehension over declared data monads, with programmable aggregation and explicit cost

L.G. Meredith
F1R3FLY.io

26 October 2026 — draft 2

Abstract

f1r3|@ng (spoken *f1r3lang*; formerly MeTTaIL, and before that rholang 1.4) has one comprehension, `for`, and it binds in the *space*: its sources are channels, its data are shared with concurrent writers, and when several candidate matchings are available it resolves them by choosing one. This note adds a second comprehension,

```
foreach( ptrn <- mndInst where cond ) yield( rslt ) P
```

which binds in a *declared data monad*: its source is a term of a monadic sort, it meets no writer, and it resolves its candidates by aggregating all of them. Every surviving binding contributes the normal form of its body, the contributions are combined, and the result is published once, on the channel `rslt`, for any process holding that channel to consume.

The design has four parts. In the data definition sublanguage, a `Monads` builder declares that a sort is a monad, gives unit and flatMap as rewrites, and supplies a monad morphism toBag into finite bags, whose size is the *measure* that every cost bound in the note rests on. The monad laws are proof obligations, not equations, so matching modulo a theory never meets higher-order unification. In the control flow sublanguage, bodies are evaluated by *lookahead*: each is reduced to normal form, in isolation, within a terminating fragment picked out by typing. The comprehension is then a single rewrite whose reduct is one output, which is the precise sense in which it is synchronous, and which is what entitles it to stand on the left of `;`. Aggregation is *programmable*: a `by` clause names a theory instance descending from `CommutativeMonoid`, checked by origin, and the default is the `ParMonoid` of parallel composition. Because parallel composition is the free commutative monoid, every other aggregator is defined by the universal property, and accumulation can be tree-shaped without changing the meaning. In the cost-accounted surface the comprehension carries two gates, an outer one charged per binding and an inner one funding each body's lookahead, and a linear funding proof needs a bound on the measure of the instance.

A worked example declares the parametric rho calculus — the syntax of the calculus as a type constructor in its data and its variables — as a syntax monad whose bind is semantic substitution, with the group action of oriented processes as a distributive law and with symmetry reducing the precharge of a comprehension over a symmetric template.

`foreach` is a conservative extension of the core, so it need not be implemented by translation into `for`. The translation is retained as the reference semantics, and a native implementation is correct when it is *gradedly* bisimilar to it — the same behaviour and the same bill.

Contents

1 Two comprehensions **3**

2 Background **4**
2.1 The data definition sublanguage . . . 5
2.2 The control flow sublanguage . . . 5
2.3 The cost-accounted surface . . . 6
2.4 Monads one level up . . . 6

3 Declaring a data monad **6**
3.1 The prerequisite: a sort constructor . . . 6
3.2 The `Monads` builder . . . 7
3.3 Monads and the algebra of theories . . . 9

4 The `foreach` comprehension **9**
4.1 Syntax . . . 9
4.2 Meaning in one rule . . . 9
4.3 Static discipline . . . 10
4.4 Groups: `;` and `&` . . . 11
4.5 `where` in `foreach` . . . 11
4.6 Lookahead and the terminating fragment . . . 11
4.7 Reference semantics . . . 12
4.8 Synchrony . . . 13

5 Aggregation **13**
5.1 Parallel composition is the free commutative monoid . . . 13
5.2 Aggregators are theory instances . . . 13
5.3 Tree-shaped accumulation . . . 15
5.4 The staged path . . . 15

6 Sequencing **16**

7 Cost **17**
7.1 Two gates . . . 17
7.2 The atomic outer gate . . . 18
7.3 Precharge and linear funding proofs . . . 18

8 Implementation **19**
8.1 The translation is the specification, not the strategy . . . 19
8.2 What native evaluation reuses . . . 20
8.3 What native evaluation must not inherit . . . 20
8.4 What changes in the representation . . . 20
8.5 What gets simpler . . . 20
8.6 Built-in and declared monads . . . 21
8.7 Where the frontend stands . . . 21

9 Interactions with the rest of f1r3|@ng **21**
9.1 Channel types . . . 21
9.2 Generated logics . . . 21
9.3 Graded `where` clauses . . . 22
9.4 Simulation . . . 22
9.5 The quantum fence . . . 22
9.6 Summary . . . 22

10 A worked example: the parametric rho calculus **23**
10.1 The construction . . . 23
10.2 The syntax monad . . . 23
10.3 The functor in D . . . 25
10.4 Orientation is a distributive law . . . 25
10.5 The capabilities . . . 26
10.6 `foreach` over a process . . . 27
10.7 Symmetry reduces the precharge . . . 27
10.8 Dynamic frames, and where the word problem lands . . . 28

11 A first tranche **28**

12 Open questions **28**

1 Two comprehensions

The comprehension is the shape data access takes when written down honestly: name a source, filter it, project the survivors, act. SQL, LINQ [9], XQuery's FLWOR and the list comprehension [11] are the same shape, and Wadler's observation that the comprehension is monadic [12], in the sense Moggi gave to notions of computation [10], is what makes the family a family: every one of them is a notation for flatMap in some monad.

f1r3|@ng's receipt

```
for( ptrn <- chan where cond ) P
```

is that shape with the `DO` made first-class [15]. But the monad it binds in is unusual. The source is a channel; the datum is produced by a concurrently running writer; the join `&` is a rendezvous; the communication rule substitutes the datum into the continuation and thereby *witnesses* the meeting. Call this the *space* monad. Its bind is asynchronous, it is shared, and it is nondeterministic in the strong sense that competing readers race.

Developers also need the other kind of comprehension: the one that LINQ, and Scala's `for` with `yield`, provide over a collection already in hand. A term received on a channel may itself be a list, a bag, a tree of findings, a distribution. Walking it with `for` is possible only by first spilling it onto a channel, which is both expensive and wrong: it exposes a private traversal to every other reader of that channel, and it turns a deterministic computation over a value into a race.

`foreach` is the missing comprehension. It binds in a *data monad* declared in the data definition sublanguage (DDL), evaluates a body for every surviving binding, combines the results, and publishes the combination once:

```
foreach( i <- { 1, 3, 5, 7, 9 } where i > 4 ) yield( rslt ) i
  ~> rslt!( 5 | 7 | 9 )
```

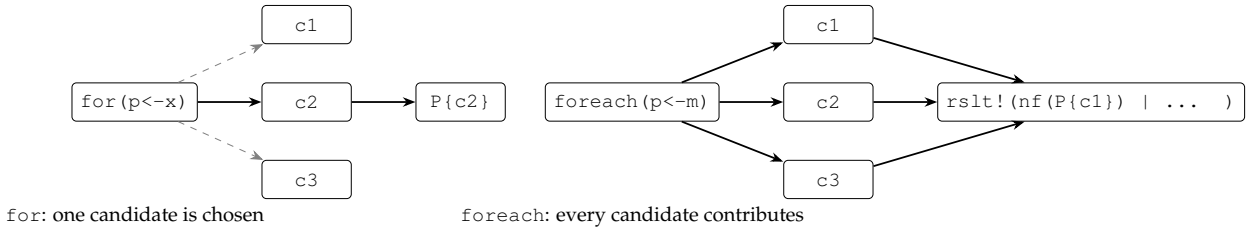


Figure 1: The two resolution disciplines for candidates. Under the default aggregator the published result is a parallel composition, because `PPar` is a bag (§5.1).

The result is on a channel because a result is something other processes consume, and in f1r3|@ng that is what channels are for. A consumer may take it, persist on it, or peek at it with the three arrows of `for`.

Choose versus sum. The cleanest way to see the difference is through the word the developer document uses for the thread running through f1r3|@ng's semantics: *candidate* [15]. A candidate is one way of assigning data to the patterns of a comprehension.

- In `for`, candidates are resolved by *choice*. One fires; the others do not.
- In `foreach`, candidates are resolved by *aggregation*. Every candidate contributes — a rejected one contributes the aggregator's unit — and the contributions are combined.

Figure 1 draws the distinction. Section 9.3 shows that it is exactly the distinction the graded `where` clause needs.

Contributions.

1. A `Monads` builder for the DDL, with the laws as obligations and a monad morphism into finite bags that both drives the comprehension and measures it (§3).
2. The `foreach` comprehension: its syntax, its meaning as a single rewrite, evaluation of bodies by lookahead in a terminating fragment, and a reference translation into the core (§4).
3. Programmable aggregation: aggregators as theory instances descending from the universal-algebra theory of commutative monoids, defined by the universal property of parallel composition, accumulated as a tree, and extended in three further stages (§5).
4. Sequencing with `;`, on the strength of termination (§6).
5. An explicit cost-accounted surface with two gates, per-binding charging, an atomic outer gate, and a precharge discipline for linear funding proofs (§7).
6. An implementation stance — native evaluation, the translation retained as specification, graded bisimulation as the correctness criterion — and a map of what changes in the node (§8).
7. An account of the interaction with the rest of f1r3|@ng (§9), a worked example on the parametric rho calculus (§10), and a first tranche (§11).

2 Background

This section fixes the parts of f1r3|@ng the feature touches. Fuller treatments are in the developer document [15] and the GSLT reference [17]. f1r3|@ng is built on the reflective higher-order rho calculus [25], presented in its own DDL as one theory among others.

2.1 The data definition sublanguage

A language definition is a `Theory`. Its body is an ordered chain of postfix builders — `Types`, `Exports`, `Replacements`, `Terms`, `Equations`, `Rewrites` — applied to a starting theory, and theories combine by join, meet and difference over shared ancestors [26]. Every element introduced by a builder carries an origin token, and two elements are identified exactly when their origins agree; this is what makes a join a pushout rather than a union, and what produces the `[join-collision]` and `[unshared-name]` diagnostics. A replacement renames an element and keeps its origin.

Term constructors are written in judgement form,

```
PInput . n:Name, ^x.p:[Name -> Proc] |- n "?" x "." "{" p "}" : Proc;
PPar . ps:HashBag(Proc) |- "{" ps.*sep("|") "}" : Proc;
```

with binders `^x.p` as first-class arguments, and three built-in collection sorts `HashBag`, `Set` and `List` over a category. There are no user-declared sort constructors: in the reference elaborator a sort is either a category or a built-in collection of one. That fact governs the first tranche of this design.

The reference corpus includes a universal-algebra module whose tower of theories, from `EmptySet` and `Monoid` through `CommutativeMonoid` and `Group` to `Rig` and `Ring`, is built by parameters and replacements, and the rho calculus's own parallel composition is defined as an instance of it:

```
Theory ParMonoid(cm: u.CommutativeMonoid) {
  cm
    Exports { Elem => Proc; }
    Replacements { Zero => PZero . |- "0" : Proc;
                   Plus => PPar . ps:HashBag(Proc) |- "{" ps.*sep("|") "}" : Proc; }
}
```

That definition is the hinge of §5.

2.2 The control flow sublanguage

The comprehension has the form

```
for( ptrn_11 <- chan_11 & ... & ptrn_1n <- chan_1n where cond_1 ;
    ... ;
    ptrn_k1 <- chan_k1 & ... & ptrn_km <- chan_km where cond_k ) P
```

with `&` for simultaneity within a group, `;` for sequence between groups, and a `where` per group that may mention any variable bound in that group or an earlier one. Three arrows distinguish a consuming read `<-`, a persistent read `<=` and a peek `«-`. A failed guard leaves the datum in the space: a failed transaction is not rolled back, it never began [15].

Rholang 1.1 also provides a synchronous send, `x!?(Q); K`, which the node's normaliser desugars as

```
new r in { x!(*r, Q) | for( _ <- r ){ K } }
```

so that `K` runs once the receiver on `x` acknowledges on the fresh name `r` [27]. Chains `x1!?(Q1); ...; xn!?(Qn); K` are nested instances. The continuation runs only if the receiver chooses to reply.

2.3 The cost-accounted surface

Cost accounting is written in the language rather than bolted onto an interpreter [13]. A signed term $\{P\}_s$, concretely `{% P %}[s]`, runs `P` once a token for signature `s` is available; a budget is a stack `s :: s :: ... :: ()` whose depth is the number of operations it funds. Signatures are ground names, sections `# P`, and compounds. The surface already admits per-clause signed binds,

```
{% for( {% y1 <- x1 %}[s1] & {% y2 <- x2 %}[s2] ) P %}[ s0 ]
```

in which each rendezvous surface is funded independently, and in the native four-sort grammar the continuation and payload slots of `for` and send have the signed-term sort $\mathbb{T}$. That sorting — *wrapping by construction* — is what makes subject reduction coincide with the absence of unmetered execution [14]. Funded runs are bounded: strong normalisation holds conditionally on funding, and the development records uniqueness of normal forms on the funded fragment [27]. Both facts are used in §4.6.

2.4 Monads one level up

The GSLT reference treats three monads: the cost monad C, the history monad H and the Hypercube construction Hyp [17, 14, 18, 23]. All three are endofunctors on the category of theories: they take a theory and return a metered, logged or typed theory.

The monads of this note live one level down. A declaration "M is a monad" is a statement *inside* a theory, about one of its sorts. The distinction matters in two places. In prose, the phrase "the monad" becomes ambiguous; this note says *data monad* for the new kind and *theory-level monad* for C, H and Hyp. And in semantics, the theory-level monads act on theories that contain data monads, which raises the question of whether the declaration survives them.

Proposition 2.1 (Laws modulo erasure)**.** *Let theory T declare a data monad* M *whose laws hold in T. Then $\mathsf{C}(T)$ and $\mathsf{H}(T)$ carry the declaration, and the laws hold in each up to the corresponding erasure back to T, but not in general on the nose.*

Sketch. The two sides of the left unit law, $\mathsf{flatMap}(\mathsf{unit}(a), k)$ and $k\,a$, differ by one application of the $\mathsf{flatMap}$ rewrite. In $\mathsf{C}(T)$ that application is gated and consumes a token; in $\mathsf{H}(T)$ it is logged as an event. The two sides therefore differ in cost and in history, and agree after the erasures $\mathsf{C}(T) \to T$ and $\mathsf{H}(T) \to T$ studied in [19]. The right unit and associativity laws are similar. □

This is the expected behaviour, not a defect, and it anticipates the correctness criterion of §8: once cost is explicit, "the same computation" means the same behaviour *and* the same bill.

3 Declaring a data monad

3.1 The prerequisite: a sort constructor

A monad is a sort constructor $\mathsf{M}(-)$ with structure. map alone needs two element sorts, $(A \to B) \to \mathsf{M}(A) \to \mathsf{M}(B)$. The DDL has no user-declared sort constructors, so the design proceeds in two steps.

Step 1: the built-in collections. `List`, `Set` and `HashBag` are already sort constructors, and they are the canonical monads: the list monad, the finite powerset monad, and the finite multiset monad. Declaring them monadic needs no new sort machinery. They are declared once, in a prelude supplied by the toolchain, and are available to every theory.

Step 2: sort parameters. The general feature extends `Types` with parameterised categories,

```
Types { Tree(A); }
```

where `A` ranges over categories. This is the real feature, and it is independently useful: the Hypercube construction currently generates type systems over unparameterised sorts [23]. Its motivating instance is the calculus itself, whose syntax is already treated in the working notes as a type constructor in its data and its variables; §10 works it through.

A rejected alternative. A theory parameter can stand in for a type parameter — `Theory List(e: EmptySet)` is a list of whatever `e` exports. But then M is a map from theories to theories, the feature climbs back to the level of C and H, and the sharing discipline objects: `List(List(s))` reintroduces the list constructors along its own chain and draws `[repeat-label]`. The diagnostic is correct, and the alternative is rejected.

3.2 The `Monads` builder

A new postfix builder declares one or more data monads. Its entries name the monad, the sort it is a monad on, the interpretation of the operations, and the structure it offers to the comprehension. The prelude declarations of the list and multiset monads are written with a sort variable `A` that Step 2 makes available to users and that the prelude may use from the start.

```
Monads {
  Seq over List {
    unit : (a) ~> [a];
    flatMap : ([], ^x.k) ~> [];
    flatMap : ([a, ...r], ^x.k) ~> (Concat (subst ^x.k a) (flatMap r ^x.k));
    toBag : ([a, ...r]) ~> {a, ...(toBag r)};
    ordered;
    laws : LeftUnit, RightUnit, Assoc, ToBagMorphism;
  }
  Bag over HashBag {
    unit : (a) ~> {a};
    flatMap : ({}, ^x.k) ~> {};
    flatMap : ({a, ...r}, ^x.k) ~> (Union (subst ^x.k a) (flatMap r ^x.k));
    toBag : identity;
    commutative;
    laws : LeftUnit, RightUnit, Assoc, Comm, ToBagMorphism;
  }
}
```

The surface is illustrative; its content is not. Five decisions are embedded in it.

The interpretation is a set of rewrites. The DDL defines operations only through rules, so unit and flatMap are given by cases over the sort's constructors, with the continuation as a first-class abstraction `^x.k` of sort `[A -> M(B)]`. These are ordinary rewrites: they are oriented, they fire, and they "wiggle" like any other rule of the theory. map may be given explicitly for efficiency; when it is, $\mathsf{map}\ f = \mathsf{flatMap}(-, \mathsf{unit} \circ f)$ becomes one more obligation.

The laws are obligations, not equations. Matching in f1r3|@ng is matching modulo the equations of the theory that types the channel. Placing $\mathsf{flatMap}(m, \lambda x.\mathsf{unit}(x)) = m$ in an `Equations` block would put the matcher into higher-order unification, which is undecidable in general. The `laws` entry therefore *names* the laws; the elaborator records them as obligations, discharged by proof (as the cost monad's laws are, in Rocq and Lean) or by bounded exhaustive checking (as in the Sage harness), and never hands them to the matcher.

Obligation 3.1 (Monad laws)**.** For every declared data monad, the left unit, right unit and associativity laws hold up to the equations of the theory.

Obligation 3.2 (Confluence with the host theory)**.** The flatMap and toBag rewrites have no critical pair with the other rewrites of the theory, modulo its equations, that fails to join.

toBag **is what the comprehension uses.** A comprehension visits the elements of its source with their multiplicities. That is a natural transformation toBag : M $\Rightarrow$ HashBag, the support with multiplicity, and it is the one piece of structure every monad used with `foreach` must supply. For dependent groups (§4.4) to agree with the monad's own sequencing it must be a monad morphism:

$$\mathsf{toBag}(\mathsf{unit}(a)) = \{a\}, \qquad \mathsf{toBag}(\mathsf{flatMap}(m, k)) = \biguplus_{x \in \mathsf{toBag}(m)} \mathsf{toBag}(k\,x).$$

It is one for `Seq` (forget order) and `Bag` (identity). It is not one for the finite powerset, whose flatten removes duplicates; for `Set` the comprehension therefore visits *binding instances*, and deduplication of results is a choice of aggregator, not of source (§5.4).

Obligation 3.3 (Bag morphism)**.** toBag is a monad morphism M $\Rightarrow$ HashBag.

The *measure* of an instance is the size of its bag,

$$\mathsf{measure}(m) \;=\; |\mathsf{toBag}(m)|,$$

and it is the quantity on which the completion of the reference translation (§4.7) and every cost bound of §7 rest. Defining it this way, rather than declaring it separately, makes the earlier "exact measure" obligation a consequence of Obligation 3.3: $\mathsf{flatMap}(m, k)$ invokes k once per element of $\mathsf{toBag}(m)$.

Capabilities are claims, and each one licenses a feature. Beyond toBag, two optional entries matter, and each is checked where it is used.

Entry	Meaning	Licenses
`toBag` (required)	elements with multiplicity; a monad morphism	`foreach` over the monad at all; its measure
`ordered`	a canonical enumeration order on $\mathsf{toBag}(m)$	non-commutative (ordered) aggregators (§5.4)
`commutative`	order of independent binds is immaterial [8]	the join `&` under an ordered aggregator (§4.4)

Two capabilities the earlier draft required have gone. A zero is no longer needed for filtering, because a rejected candidate contributes the *aggregator's* unit (§4.5); and an algebra on processes is no longer needed, because the result is data published on a channel, and aggregation is the job of the aggregator (§5).

One sort may carry several monads. The monads are named (`Seq`, `Bag`) because a sort can legitimately carry more than one monad structure, just as a rig carries two monoids on one carrier. A comprehension may name the monad it binds in, `foreach[Seq](...)`; the qualifier may be omitted when exactly one monad is declared on the source's sort.

3.3 Monads and the algebra of theories

A `Monads` entry is an element like any other, so it receives an origin token at introduction and is carried through inheritance.

- **Join.** Two theories that inherit `Seq` from one shared instance join cleanly, and the comprehension sees one monad. Two theories that independently declare a monad under the same name on the same sort draw `[join-collision]`: they are two structures with one spelling.
- **Difference.** Subtracting a constructor that a monad's rules mention leaves the declaration dangling. That is a new diagnostic, `[dangling-monad]`, and it is the analogue of the existing `[forward-reference]`.
- **Ordering.** A `Monads` block must follow the `Terms` blocks whose labels its rules mention, by the existing chain discipline.
- **Replacements.** Renaming a constructor preserves its identity and therefore preserves the monad. Replacing a constructor by one of different arity reproduces the open arity-drift problem already recorded for `PPar`: the inherited flatMap rules are written over a shape that no longer exists. Whatever resolution is adopted there — arity-preserving replacements, or a way to declare inherited rules structural — must be adopted here as well.
- **Parameters.** Once Step 2 lands, monad transformers are theories that take a monad as a parameter. This is the fourth rung of the DDL doing what it was built for.

4 The `foreach` comprehension

4.1 Syntax

```
Proc ::= ... | Foreach | Foreach ";" Proc
Foreach ::= "foreach" Qual? "(" Groups ")" "yield" "(" Name ")" By? Proc
Qual ::= "[" MonadName "]"
Groups ::= Group ( ";" Group )*
Group ::= Bind ( "&" Bind )* ( "where" Cond )?
Bind ::= Pattern "<-" Term -- Term of a monadic sort
By ::= "by" TheoryInstance -- default: ParMonoid
```

The surface coincides with `for` wherever the meanings coincide and diverges visibly where they do not. The channel named by `yield` is free in the comprehension and is where the result appears. The body `P` is in the scope of the pattern variables. Its cost-accounted form is the subject of §7; until then, gates are elided.

4.2 Meaning in one rule

For a single group,

$$\texttt{foreach(}\ p \leftarrow m\ \texttt{where}\ c\ \texttt{)}\ \texttt{yield(}\ r\ \texttt{)}\ \texttt{by}\ A\ P$$
$$\rightsquigarrow\ r!\Big(\ \bigoplus_A\ \{\,\mathsf{contrib}(x)\ :\ x \in \mathsf{toBag}(m)\,\}\Big)$$

where $\bigoplus_A$ is the operation of the aggregator A (empty sum: its unit ε_A), and

$$\mathsf{contrib}(x) \;=\; \begin{cases} \mathrm{nf}(P\sigma) & \text{if } \sigma = \mathrm{match}(p, x) \text{ exists and } \mathrm{nf}(c\sigma) = \texttt{true}, \\ \varepsilon_A & \text{otherwise.} \end{cases}$$

nf is the normal form computed by lookahead (§4.6). Under the default aggregator $A = \texttt{ParMonoid}$, $\bigoplus$ is `|` and ε is `0`:

```
foreach( i <- { 1, 3, 5, 7, 9 } where i > 4 ) yield( rslt ) i ~> rslt!( 5 | 7 | 9 )
foreach( i <- { 1, 3, 5, 7, 9 } where i > 4 ) yield( rslt ) i * i ~> rslt!( 25 | 49 |
    81 )
foreach( i <- { 1, 3 } where i > 4 ) yield( rslt ) i ~> rslt!( 0 )
```

Three readings of the rule are worth stating at once.

- *Bodies run.* Each surviving body is reduced to normal form, so its internal computation happens. What it cannot do during the comprehension is meet a process outside itself; a send or receipt on an outside channel remains, unreduced, in its normal form and so in the published result.
- *The result is data.* The publication is a send, so its payload is quoted, as every send's is. A consumer that wants the contributions to run says so: `for( @ps <- rslt ){ ps }`.
- *One step.* From outside, the comprehension is a single reduction whose reduct is a single output. This is the strongest reading of "synchronous" available, and §4.8 shows that the reference translation realises it.

Dependent groups compose the rule through flatMap (§4.4).

4.3 Static discipline

Sources are terms of a monadic sort. In `for`, a source is a channel. In `foreach`, a source is a term whose sort carries a declared monad. A channel may not appear as a `foreach` source; a monadic term may not appear as a `for` source except by being quoted into a name, in which case it is a channel and nothing more. The two comprehensions nest freely in each other's bodies. This requires something of the CFL it has not needed before: the sort of the source expression must be known, by inference from the pattern or the channel type that bound it, or from an explicit qualifier.

One arrow. Only <− is meaningful. Persistence <= has no meaning over a value, and a peek «− is moot because a term source is never consumed. Both are refused rather than reinterpreted.

One matcher per element. Matching in f1r3|@ng is modulo the equations of the theory, and a pattern can have several matchers against one element (a remainder pattern over a bag, say). The rule uses the matcher the `match` construct selects, so each element contributes exactly once. A pattern with several matchers modulo the equations makes that selection nondeterministic and places the comprehension outside the deterministic fragment of §4.6.

Typing. Write $\mathcal{T}$ for the terminating fragment of §4.6 and $|A|$ for the carrier sort of the aggregator.

$$\frac{\Gamma \vdash m : \mathsf{M}(B) \quad p : B \rightsquigarrow \Delta \quad \Gamma, \Delta \vdash_{\mathcal{T}} c : \mathsf{Bool} \quad \Gamma, \Delta \vdash_{\mathcal{T}} P : |A| \quad \Gamma \vdash r : \mathsf{chan}(|A|)}{\Gamma \vdash \texttt{foreach[}\mathsf{M}\texttt{](} p \leftarrow m \texttt{ where } c \texttt{) yield(} r \texttt{) by } A\, P : \mathsf{Proc}}$$

with M supplying toBag and A a commutative-monoid instance (§5). Under the default aggregator $|A| = \mathsf{Proc}$, so every body in $\mathcal{T}$ is well typed. Since a channel's type is a theory, $\mathsf{chan}(|A|)$ means a channel typed by a theory that exports the aggregator's carrier.

4.4 Groups: `;` and `&`

Between groups, `;` is dependent bind: in

```
| foreach( x <- m ; y <- n(x) ) yield( r ) P
```

the source of the second group may mention `x`, and the candidates are the elements of

$$\mathsf{toBag}\big(\mathsf{flatMap}(m,\ \lambda x.\,\mathsf{map}(n(x),\ \lambda y.\,(x,y)))\big)\ =\ \biguplus_{x \in \mathsf{toBag}(m)} \{\,(x,y) : y \in \mathsf{toBag}(n(x))\,\},$$

the equality being Obligation 3.3. The per-group scoping of `where` carries over unchanged.

Within a group, `&` means *independent* binds: neither source mentions the other's variables. Under a commutative aggregator — every aggregator of Stage 1 — the order in which independent binds are taken cannot show in the result, so `&` is always permitted. Under an ordered aggregator (§5.4) the order would show, and `&` is then permitted only on monads that claim `commutative`. The restriction has moved from the monad to the aggregator.

4.5 `where` in `foreach`

A rejected candidate contributes the aggregator's unit. Filtering therefore needs nothing of the monad: no zero, no MonadPlus structure. The diagnostic the earlier draft proposed for monads without a zero is withdrawn.

The same keyword now has two operational readings, and the developer document should state both:

- in `for`, a failed guard *leaves the datum in the space*;
- in `foreach`, a failed guard *contributes the unit*.

What reconciles them is that in both cases the rejected candidate is a non-event: nothing is consumed, nothing is undone, and nothing downstream can distinguish it from its absence.

4.6 Lookahead and the terminating fragment

The rule needs $\mathrm{nf}(P\sigma)$ and $\mathrm{nf}(c\sigma)$ to exist and to be unique. Bodies and guards are therefore evaluated by *lookahead*: each is reduced to normal form by itself, in isolation from the rest of the configuration, and the normal form is what is contributed. Two properties make this well defined.

Termination. Lookahead must finish. A type system picks out a terminating fragment $\mathcal{T}$ of f1r3|@ng, and the comprehension demands that its guards and bodies be well typed in it. The cost-accounted calculus already supplies one candidate: funded runs are bounded, so a body wrapped as `{% P %}[t]` with a finite budget terminates, and the depth of the budget bounds the work. The type system and the precharge of §7.3 are then the same discipline. Syntactic fragments — no replication, no recursion through the space — are another candidate, and cheaper to check.

Uniqueness. Termination is not enough. The commutativity of the aggregator makes the *order* in which contributions arrive undetectable; it does not hide *choices* made inside a body. A body with an internal race has several normal forms, and the published result is then nondeterministic in a

way a consumer can observe. So $\mathcal{T}$ must also guarantee unique normal forms. The development records uniqueness of normal forms on the funded fragment [27]; since unrestricted rho is not confluent, the hypotheses of that result must be checked against the bodies the fragment is meant to admit.

Obligation 4.1 (Terminating, deterministic fragment)**.** Every term typed in $\mathcal{T}$ has a normal form, reached in a number of steps bounded by its type, and any two normal forms are equal up to the equations of the theory.

Lookahead is what makes bodies *run* without making the comprehension wait on the world: a body cannot block on an outside writer, because during lookahead there is no outside.

4.7 Reference semantics

The rule is specified by a translation into the core, which is the standard against which any implementation is judged (§8). The source is spilled onto a private channel via toBag, every element is visited by a persistent receipt and contributes either the lookahead of its body or the unit, and contributions are folded into a private accumulator that publishes once, when a count reaches the measure:

```
[[ foreach( ptrn <- m where cond ) yield( rslt ) P ]] =
 new src, acc, ans in {
   [[ m ]]_src // src!(x1) | ... | src!(xn), n = measure(m)
 | acc!( 0, Nil )
 | if ( n == 0 ) { rslt!( Nil ) } // empty source: publish the unit
 | for( @x <= src ){
    match x {
      ptrn where cond => ans!( [[ P ]] )[*] // [*]: lookahead to normal form
      _ => ans!( Nil )
    }
   }
 | for( @(k, a) <= acc & @e <= ans ){
    if ( k + 1 == n ) { rslt!( a | e ) } else { acc!( k + 1, a | e ) }
   }
 }
```

Three features of the translation carry its correctness.

- **Initialisation.** The accumulator starts at the unit, `acc!( 0, Nil )`; without it no join fires.
- **Completion.** The accumulator is private, and `rslt` is written exactly once, when the count reaches n. A process holding `rslt` can never observe or steal a partial sum, and the remaining folds can never be stranded.
- **Every element counts.** The pattern sits in a `match` with a catch-all rather than on the receipt, so an element that fails the pattern or the guard still contributes, and the count reaches $n = \mathsf{measure}(m)$ exactly.

The measure therefore enters the *semantics*, not only the cost: it is the completion condition. For dependent groups the total is not known at entry, and the count becomes a deficit in $\mathbb{Z}$: it starts at $\mathsf{measure}(m)$, each first-group binding x adds $\mathsf{measure}(n(x)) - 1$ as it is expanded, each leaf contribution subtracts one, and publication happens at zero. Integers under addition are a commutative monoid, so the deficit is one more aggregator running alongside the first (§5.3).

The translation exhibits what the monad must supply: toBag, and nothing else, for a single group; flatMap, through Obligation 3.3, for dependent ones.

4.8 Synchrony

Proposition 4.2 (Synchrony)**.** *Suppose the guards and bodies of a* `foreach` *are typed in* $\mathcal{T}$*. Then its reference translation is weakly bisimilar to the single output* $r!(\bigoplus_A \{\mathsf{contrib}(x)\})$ *of §4.2.*

Sketch. Every communication of the translation other than the final send is on `src`, `acc` or `ans`, which the translation restricts; these are internal steps. Lookahead is internal by definition. By Obligation 4.1 each contribution exists and is unique; by commutativity and associativity of the aggregator, the value accumulated is independent of the order of the folds; the count guarantees exactly one publication. A finite burst of internal steps with a unique end-point followed by one visible output is weakly bisimilar to that output. □

So `foreach` is synchronous in the strongest available sense: it *is* one step up to weak bisimulation. Termination comes from $\mathcal{T}$ (and from funding, §7); determinism from $\mathcal{T}$, from Obligation 3.2 for the monad's rewrites, and from the aggregator's laws. The slogan — concurrent internally, sequential externally — is the content of the proposition.

5 Aggregation

5.1 Parallel composition is the free commutative monoid

`PPar` *is* `HashBag(Proc)`, and a finite bag is the free commutative monoid on its elements. So for every commutative monoid $(A, \oplus, \varepsilon)$ and every assignment $h : \mathsf{Proc} \to |A|$ there is exactly one monoid homomorphism $\hat{h}$ from parallel composition to A,

$$\hat{h}(P \mid Q) = \hat{h}(P) \oplus \hat{h}(Q), \qquad \hat{h}(\mathtt{0}) = \varepsilon.$$

Programmable aggregation is *defined* by this universal property. The default rule of §4.2 is the meaning of `foreach`, and every other aggregator is post-composition with a homomorphism out of it:

$$\texttt{foreach(...)}\quad \texttt{yield(} r \texttt{) by } A\ P \quad \text{means} \quad r!\big(\hat{h}\,(\Pi\,\mathsf{contrib})\big).$$

The assignment h needs no syntax. The body is typed in the carrier, so the body *is* h: it maps each binding to an element of $|A|$.

Three consequences follow, and they are the reason to choose this path.

- **No new semantics.** Parallel composition stays the reference meaning; aggregators add structure on the output side only.
- **Fusion is a theorem.** An implementation folds each contribution into the accumulator as it arrives instead of building the parallel composition and then folding it. That is correct exactly because $\hat{h}$ is a homomorphism, and the specification never has to mention it.
- **Order stays invisible.** The argument that reordering by the concurrency is undetectable — made first for parallel composition — holds for every commutative aggregator, for the same reason.

5.2 Aggregators are theory instances

The `by` clause names a *theory instance*: an elaborated theory expression — an application or a `let`-bound name, of the grammar theory bodies already use — whose presentation descends from `CommutativeMonoid`. Integer summation is one replacement away from the universal-algebra module:

```
Module Arith {
  import "UnivAlg.module" as u
  Theory IntSum() {
    let s = u.EmptySet() in (
    let m = u.Monoid(s) in (
    let cm = u.CommutativeMonoid(m) in (
    cm
      Exports { Elem => Int; }
      Replacements { Zero => IZero . |- "0" : Int;
                     Plus => IAdd . x:Int, y:Int |- x "+" y : Int; }
      Rewrites { IAddEval : (IAdd n k) ~> [n + k]; // host integer primitive
    }))
  }
}
```

and in a program,

```
import "Arith.module" as a

foreach( i <- { 1, 3, 5, 7, 9 } where i > 4 ) yield( rslt ) by a.IntSum() i
  ~> rslt!( 21 )
```

The default is an instance too. Without a `by` clause, the aggregator is the `ParMonoid` instance of the theory that defines `PPar` in the program's own language. Nothing about the default is special-cased; it is the instance that happens to be free.

Commutativity is checked by origin, not by name. Instances were chosen over named operations because of what the elaborator can then decide. Every element of an elaborated presentation carries its origin token, and a replacement keeps the origin of what it renames. So the elaborator can check that the instance's operation *is* the `Plus` introduced by `u.CommutativeMonoid`, carrying that theory's associativity, unit and commutativity equations by origin, and not merely an operation called `Plus`. This is the mechanism that already distinguishes a shared `Name` from two categories that happen to share a spelling. Two diagnostics follow.

- `[aggregator-not-commutative]`: the instance's operation does not descend from `CommutativeMonoid`'s `Plus`. An operation that is associative but not known to be commutative is an ordered aggregator (§5.4) or nothing.
- `[aggregator-ambiguous]`: the instance carries more than one commutative-monoid structure on its carrier — a rig carries two, by construction. The clause must then select one by the parameter that supplied it, `by r.add`.

Elaboration is compile time. Instances are elaborated by the client toolchain, as all theory expressions are. An aggregator is therefore fixed by the program text; it cannot be chosen by data received at run time. That is the right default for a construct whose cost must be bounded before admission.

What remains an obligation. Origins establish that the operation is *declared* associative, commutative and unital. That the rewrites which *compute* it respect those equations — that `IAddEval` agrees with commutativity, say — is a confluence obligation on the instance, of the same kind as Obligation 3.2, and the operation's own evaluation must lie in $\mathcal{T}$.

5.3 Tree-shaped accumulation

With a commutative monoid, the accumulator need not be a single serial channel. Let every contribution be a singleton partial result, paired with a count, and let partial results combine wherever they meet:

```
new src, acc in {
  [[ m ]]_src
| for( @x <= src ){
   match x {
     ptrn where cond => acc!( 1, [[ P ]] )[*]
     _ => acc!( 1, eps )
   }
  }
| for( @(k1, a1) <= acc & @(k2, a2) <= acc where k1 + k2 < n ){ acc!( k1 + k2, a1 (+)
    a2 )[*] }
| for( @(k1, a1) <- acc & @(k2, a2) <- acc where k1 + k2 == n ){ rslt!( a1 (+) a2 )
   [*] }
}
```

with `(+)` and `eps` the aggregator's operation and unit, and two further rules for $n \leq 1$: an empty source publishes `eps`, and a single contribution is published as it stands. Three observations.

- The pair of a count and a value lives in the product $(\mathbb{N}, +, 0) \times (A, \oplus, \varepsilon)$, itself a commutative monoid. The completion test is a second aggregator running beside the first, and the deficit of §4.7 for dependent groups is the same construction over $\mathbb{Z}$.
- Combination is no longer a chain of length n; partial results merge pairwise wherever they meet, so the depth can be logarithmic. This is the combiner of MapReduce [2], and it is the shape of parallelism the GPU deployment of the node wants.
- Accumulation is itself a rewrite system on a bag of partial results, in the style of the communication rule, and it publishes exactly once.

The serial translation of §4.7 and this one are weakly bisimilar by commutativity and associativity, and either may serve as the reference.

5.4 The staged path

Each further stage adds one algebraic requirement on the aggregator, checked by origin in the same way, and costed by the same per-combine rule.

Stage	Aggregator descends from	Examples	Requirement on the source
1	`CommutativeMonoid`	parallel composition, sums, counts, products	`toBag`
2	a `Semilattice`: commutative, idempotent	union, maximum, boolean or	`toBag`
3	`Monoid` only	lists, strings, function composition	`toBag` and `ordered`; `&` needs `commutative`
4	a $\mathcal{V}$-`Semimodule`	weighted bags, expectations, amplitudes	`toBag`; a graded `where`

Stage 2: idempotent aggregators. A semilattice is a commutative monoid with $a \oplus a = a$. It needs a theory that the universal-algebra module does not yet contain: a `Semilattice`, taking a

commutative monoid as its parameter and adding the idempotence equation. Its first use settles a question the earlier draft left open: whether a `Set` source should deduplicate. It need not. The comprehension visits binding instances (§3), and

```
foreach( x <- { 1, 2 } ; y <- { 3 } ) yield( r ) y ~> r!( 3 | 3 )
foreach( x <- { 1, 2 } ; y <- { 3 } ) yield( r ) by u.SetUnion() y ~> r!( Set(3) )
```

Deduplication is the choice of an idempotent aggregator. Semilattices are also the coalition-logic container of the generated logics [15], so this stage is where aggregation and observation meet.

Stage 3: ordered aggregators. A monoid that is not commutative — list concatenation, string building, composition of functions — breaks the argument that order is invisible. Contributions are then tagged with their position in the source's `ordered` enumeration and collected in the monoid of finite maps from positions under disjoint union, which *is* commutative; the non-commutative fold is applied once, in index order, at publication. Tree-shaped accumulation is preserved. The source must declare `ordered`: `Seq` does, and `Bag` and `Set` have no canonical order, so an ordered aggregator over them is refused with `[aggregator-needs-order]`. This stage is also where Scala's `for/yield` lives: yielding into the source monad itself, with `unit` and concatenation, preserves its order.

Stage 4: semimodules. A graded `where` clause takes values in a resolution algebra $\mathcal{V}$ [16]. Its weight cannot be applied to a contribution unless the aggregator admits scalars: a $\mathcal{V}$-semimodule, a commutative monoid with an action of $\mathcal{V}$ distributing over $\oplus$. The graded contribution is $w \cdot \mathrm{nf}(P\sigma)$. The free $\mathcal{V}$-semimodule on processes is the weighted bag, in exactly the sense that parallel composition is the free commutative monoid, so Stage 4 is the universal-property construction of §5.1 one level up. Section 9.3 returns to it.

6 Sequencing

Rholang 1.1's synchronous send sequences on an acknowledgement that the receiver may or may not send. `foreach` has an acknowledgement it always sends: its result. So it can stand on the left of `;`, and the continuation waits on the result channel:

```
foreach( p <- m where c ) yield( r ) P ; K
  == new r in { [[ foreach( p <- m where c ) yield( r ) P ]] | for( @_ <- r ){ K } }
```

A continuation that needs the value binds it instead,

```
foreach( p <- m where c ) yield( r ) P | for( @ys <- r ){ K }
```

so no separate value form of the comprehension is needed: the acknowledgement and the result are the same datum. In the first form `r` is bound by the sugar, and is private to the comprehension and its continuation.

Why `foreach` has the right. A process may appear on the left of `;` only if the continuation is guaranteed to be released. The synchronous send cannot offer that guarantee: it depends on a convention honoured by the receiver. `foreach` can. Its guards and bodies terminate by Obligation 4.1, its count reaches the measure, and so its publication is certain whenever it is funded. Termination is what earns the place on the left of `;`, and it is the type system that grants it.

Chains. Mixed chains need no further machinery. The chain

```
x!?(Q); foreach( y <- m ) yield( r ) P; z!?(R); K
```

is nested `new`/`for` pairs, and `;` means "then" throughout: between receipt groups, after a synchronous send, and after a `foreach`. The earlier draft's sequenceable fragment of bodies is no longer needed: completion is defined by publication, for every comprehension.

7 Cost

7.1 Two gates

The cost-accounted form of the comprehension is

```
{% foreach( ptrn <- mndInst where cond ) yield( rslt ) by A {% P %}[t] %}[s]
```

and its two gates fund different things.

The outer gate is per binding. In the core, a gate funds one event, the communication. In `foreach`, the event the comprehension witnesses is each *binding*, so `[s]` funds one token per candidate — every element of $\mathsf{toBag}(m)$, whether or not its pattern and guard succeed — plus the combinations of the accumulator and the one publication. The principle is the one the developer document states, the witness is where cost attaches, with a different multiplicity.

The inner gate funds lookahead. `{% P %}[t]` funds the reduction of a surviving body to normal form. Because lookahead runs in isolation, the inner gate is spent during the comprehension, not by a later consumer, and its budget depth bounds the work: it is the terminating fragment of §4.6 made concrete. Lookahead of the guard and of the aggregator's operation is charged in the same way, on the lane the program assigns.

Per-group gates. The per-clause signed binds of the core carry over:

```
{% foreach( {% x <- m %}[s1] ; {% y <- n(x) %}[s2] ) yield( r ) {% P %}[t] %}[s0]
```

funds the bindings of each group from its own lane. This matters for dependent groups, whose second-group candidates may be supplied by a different party.

The syntax is forced. In the native four-sort grammar, continuation and payload slots have the signed-term sort $\mathbb{T}$. Giving `foreach` the same treatment — the comprehension in $\mathbb{T}$, the body in $\mathbb{T}$ — produces exactly the surface above. An unwrapped `foreach` cannot be written in the cost-accounted surface, and wrapping by construction continues to hold. The continuation after `;` is signed too, exactly as the continuation inside a desugared `!?` is:

```
{% foreach( ptrn <- mndInst ) yield( r ) {% P %}[t] %}[s] ; {% K %}[u]
```

Proposition 7.1 (The outer cost does not depend on the filter)**.** *The number of tokens a* `foreach` *draws on its outer gate for bindings equals* $\mathsf{measure}(m)$*, whether or not patterns and guards succeed. The number of body lookaheads it funds on its inner gate is the number of survivors, and is bounded by the same count.*

Proof. Every element of $\mathsf{toBag}(m)$ is matched and guarded, and contributes — its body's normal form or the unit — so every element passes the outer gate. Only survivors are looked ahead, and survivors are a subset of elements. □

The proposition closes a hole that would otherwise exist. If only the inner gate charged, a `foreach` over a very large collection with a guard that is always false would cost almost nothing, and the traversal would be free. Per-binding charging prices every candidate, and a single estimate bounds both lanes.

7.2 The atomic outer gate

The number of candidates $n = \mathsf{measure}(m)$ is computable when the gate opens, so for a single group the outer gate can be an n-ary join on the purse: one communication that takes everything the binding lane needs or takes nothing.

- If the purse holds enough, the join fires, the comprehension runs as a burst of internal steps, and it ends in its one publication.
- If it does not, nothing is consumed, and the comprehension is a non-event — exactly as an unfunded communication is in the core.

Because bodies run in isolation, the comprehension has no effects other than its publication, so on the outer lane it is all-or-nothing in the strongest sense.

The inner lane is different. A body whose lookahead exhausts its `t` budget has no normal form within budget, so it never contributes, the count never reaches n, and nothing is published. To consumers that is a non-event, but the outer tokens are spent. A funding proof that promises a result must therefore fund the inner lane in full, and the two-lane atomicity question — reserve the inner depth for every candidate at entry, or let a failed lookahead burn what the outer gate drew — is recorded as Question 12.3. Dependent groups reopen the outer question too, since their total is known only as the first group expands (Question 12.4).

7.3 Precharge and linear funding proofs

Per-binding charging moves the variability of `foreach`'s cost out of the semantics and into estimation. A linear funding proof must show that every gate a deploy will meet is funded, so for a `foreach` it needs an upper bound on $\mathsf{measure}(m)$.

Remark 7.2 (Two uses of "precharge"). In the node, *precharge* also names the escrow mechanism — precharge, run, refund — that the cost-accounting work removed in favour of gate-funded execution. That mechanism is not being reintroduced. Here *precharge* means the bound that a static funding proof assumes, and nothing is escrowed: the n-ary gate takes exactly what it needs when it fires.

What is estimated. For a single group with $n = \mathsf{measure}(m)$, an aggregator whose operation costs at most $c_{\oplus}$ to look ahead, and a body budget of depth d_t,

$$\text{outer} \;\le\; n \;+\; (n-1)\,(1 + c_{\oplus}) \;+\; 1, \qquad \text{inner} \;\le\; n \cdot d_t,$$

counting one token per binding, one per combination plus its operation, and one for the publication. For dependent groups, n is $\mathsf{measure}(m) + \sum_x \mathsf{measure}(n(x))$. A `foreach` nested inside a body is funded from the inner lane and contributes its own bound to d_t.

Where bounds come from. In decreasing order of precision:

1. *A closed instance at preparation time.* The measure is computed and the bound is exact.
2. *An instance bound by an earlier receipt or match.* Theory types say nothing about size, and a pattern such as `[a, b, ...rest]` supplies only a lower bound. The guard on that binding supplies the upper bound:

   ```
   for( @m <- ch where m.size() <= 64 ) {
     {% foreach( x <- m ) yield( r ) {% P %}[t] %}[s]
   }
   ```

   Here the `where` clause does double duty: it is the developer's filter, and it is the premise the estimator reads to discharge the precharge obligation. The honest default is to require such a guard, or a declared bound, before a `foreach` over received data may appear in a deploy that must be proven funded.
3. *At the gate.* The exact measure is available when the outer gate opens, and the n-ary gate uses it. This complements the static proof rather than replacing it: admission still needs the bound.

Two admissibility classes. A monad whose toBag is not finite on its instances — lazy streams, say — cannot be measured, and so can never be precharged, and its comprehension can never complete. Since the completion condition of §4.7 is the measure, such monads are not merely unprovable but unusable with `foreach`; the elaborator reports `[monad-unmeasured]`. Among finite monads the split is between instances with a bound available to the funding proof, which are *precharge-admissible*, and the rest, which run only under a gate that happens to be funded.

Lanes. The `s` and `t` gates draw on different per-signature purses, so the proof splits by signature, and the node's disjoint per-signature lane pools already enforce the split. If `t` may mention variables bound by the pattern — `{% P %}[#x]` — then each element pays for its own processing, which is useful when the elements of a collection come from different signers; the inner lanes then become data-dependent, and the proof must quantify over the elements' signatures rather than over a fixed lane (Question 12.5).

8 Implementation

8.1 The translation is the specification, not the strategy

Proposition 8.1 (Conservativity)**.** *A program that does not use `foreach` has the same reductions, the same costs and the same observable behaviour with the extension as without it.*

Sketch. Every new rule has a `foreach` constructor on its left-hand side, and no existing rule produces one. The `Monads` builder adds rules only to theories that declare it. The cost rule attaches only to the new gate. □

Nothing therefore obliges an implementation to compile `foreach` into `for`. The translations of §4.7 and §5.3 are retained as the reference semantics, and an implementation may evaluate `foreach` natively, as the single rule of §4.2.

Obligation 8.2 (Graded refinement)**.** The native evaluation of every `foreach` is weakly bisimilar to its reference translation under the graded, signature-labelled transition relation.

The native charge is designed to equal the reference translation's metered communications: one per element on `src`, one per combination on `acc`, and one for the publication. Graded bisimilarity then states both requirements at once, the same behaviour and the same bill. The relation already exists in the node's Rocq development, as the graded transition system and its Hennessy–Milner adequacy results [27]. Conservativity itself still needs checking against the reducer, an ordinary compatibility review.

8.2 What native evaluation reuses

Over a built-in collection, the native rule is an iterated match: the spatial matcher, the binders, and the `where` guards on match cases with fall-through, which are already implemented, supply the per-element pattern-and-guard step. What is new is lookahead — evaluating a body to normal form in a private sub-configuration that shares nothing with the tuple space — and the fold into the aggregator, which is the instance's own rewrite engine applied to its operation.

8.3 What native evaluation must not inherit

The free-reduction default. The node's consensus ledger counts only communication events; non-communication structural reductions — `new`, `match`, `if` — emit diagnostic events that are not charged. A natively evaluated `foreach`, being match-class work plus lookahead, would by default cost nothing however large the instance, and with host work unmetered by design, that would be free unbounded work. The per-binding charge and the lookahead budget of §7 are therefore mandatory. Both are charges on language-level events, not on host work, so they remain consistent with the decision to cap rather than meter host work. A hard protocol cap on measure sits beside them.

The static demand analysis. The funding analysis counts the communication nodes reachable in a normalised term, relies on receiving a term already desugared, and keeps an explicit seam for a front-end that delivers something else. A native `foreach` is that something else: it is not a communication node, and its contribution is data-dependent. The analysis gains a `foreach` rule, the bounds of §7.3.

8.4 What changes in the representation

The built-in collection methods (`nth`, `union`, `toList`, and so on) are first-order, and the method-call node cannot carry a binder. A native `foreach` needs a new process node carrying the groups' binders — most naturally reusing the receipt's bind-and-pattern encoding — the result channel, the body, and a reference to the aggregator instance by the digest of its elaborated presentation. That is a change to the protobuf model, and so it is consensus-visible. It should be gated on the protocol version, as the new cost-record fields are, and it must pass the compatibility review the frontend admission contract requires, since that contract freezes the model and protobuf files [27].

8.5 What gets simpler

- **Replay.** A native `foreach` touches the tuple space once, to publish. Lookahead and accumulation leave nothing in the event log, so replay recomputes them, and recomputation is deterministic by Obligation 4.1, Obligation 3.2 and the aggregator's laws.

- **Sequencing.** `K` after `;` is released by the publication. Natively, the sugar's private receipt can be fused away altogether.
- **Order-then-execute.** The internal steps race with nothing, across deploys or within one, so a `foreach` introduces no conflicts to mark by signature other than its publication, which is an ordinary send. It is order-independent and replay-deterministic by construction.

8.6 Built-in and declared monads

Two evaluation paths coexist.

- *Built-in collections* get a fast path in the reducer.
- *Monads declared in a theory* run their toBag and flatMap rules through the installed language's rewrite engine — the semantic-kernel service through which installed languages reduce terms. That work sits today under host-side semantic-work limits rather than the token ledger. The per-binding charge and the lookahead budget must cover this path too, or user-declared monads become the free route the built-ins were closed against. The same holds for aggregator operations defined by a theory's rewrites.

8.7 Where the frontend stands

On the integration branch, `Module` and `Theory` declarations are parsed structurally, the existing lowerer emits the node's process representation directly, and public activation is off [27]. The work this note adds lands in four places: the elaborator (the `Monads` builder, `toBag`, its obligations, and the origin check on aggregator instances with its diagnostics); the type checker (source sorts, the carrier of the aggregator, membership in $\mathcal{T}$); the reducer (the native node, lookahead in isolation, and the charge); and the funding analysis (the demand rule).

9 Interactions with the rest of f1r3|@ng

9.1 Channel types

A channel's type is an elaborated theory, so a `Monads` declaration travels with it: a receiver may `foreach` over a received term only when the channel's theory carries a monad on the term's sort. Symmetrically, the result channel's type must export the aggregator's carrier, so a consumer of `rslt` knows what it receives — parallel compositions under the default, integers under `IntSum`. Two agents sharing a language definition share its iteration and aggregation structure as well as its grammar.

9.2 Generated logics

The new constructors and rules are visible to the logic generated from the theory: unit, flatMap and toBag to the spatial connectives, their rewrites to the behavioural modality. The aggregator's carrier is a sort like any other, so a consumer's guard on the result is an ordinary formula over it.

There is a richer option. The logical counterpart of a monad is a *lax modality* [3, 1], written $\bigcirc\varphi$: "after binding in M, φ." A `Monads` declaration could generate $\bigcirc_{\mathsf{M}}$ in the generated logic, giving `where` clauses a way to speak about what a monadic value will deliver without enumerating it. The lax modality is a type-system-flavoured connective and raises no new adequacy question: it is consistent with the established position that Hypercube-generated logics are type systems and are not expected to be adequate.

9.3 Graded `where` clauses

Here the choose-versus-sum distinction of §1 does its work.

- In `for`, a graded guard *weights candidates in a race*: the weight changes which candidate is chosen.
- In `foreach`, there is no race. A graded guard must *scale contributions*: the weight multiplies what a candidate adds to the aggregate.

Scaling contributions requires the aggregator to be a semimodule over the value algebra — Stage 4 of §5.4. Parallel composition has no scalars, so a graded `where` under the default aggregator is refused, and the requirement falls on the aggregator, where it belongs, rather than on the monad.

9.4 Simulation

A `foreach` whose aggregator is the weighted bag, and whose guard is graded, is a stochastic simulation step: programming in the large, written with the same surface as programming in the small [24]. The "sum of all possible futures" of the equational route to a quantum reading of rho is what such a comprehension publishes.

9.5 The quantum fence

With amplitudes as the value algebra, aggregation is a sum of weighted vectors, and Kleisli composition in the vector-space monad is matrix multiplication — which is why the rho–linear-algebra correspondence and the comprehension meet. But a body that uses its bound variable twice is not linear, and the quantum reading of graded `where` clauses is restricted to the linear fragment of rho. The linearity check of that fragment applies to `foreach` bodies unchanged, and it is a further restriction on $\mathcal{T}$ for this reading.

9.6 Summary

Feature	Interaction with `foreach`
Theory-level monads	laws hold modulo erasure in $\mathsf{C}(T)$, $\mathsf{H}(T)$ (Prop. 2.1)
Equations	laws are obligations, never equations; no higher-order matching
Rewrites	interpretation and `toBag` are rewrites; confluence obligation
Algebra of theories	origin tokens; `[join-collision]`, `[dangling-monad]`; arity drift shared with `PPar`
Universal-algebra tower	aggregators are `CommutativeMonoid` instances, checked by origin; default `ParMonoid`
Arrows	only `<-` on sources; consumers use all three on the result
`;` and `&`	dependent bind via `toBag`; `&` free under commutative aggregators
`where`	rejected candidates contribute the aggregator's unit
Synchronous send	`foreach` sequences unconditionally; `!?` by convention
Cost	two gates; per-binding outer charge; lookahead budget; precharge by measure
Channel types	monad travels with the source's theory; result typed by the carrier
Generated logics	new constructors visible; optional lax modality
Graded `where`	needs a semimodule aggregator (Stage 4)
Quantum reading	linear bodies only
Order-then-execute	one publication; replay-deterministic

10 A worked example: the parametric rho calculus

The prelude monads of §3 are collections. The example that motivates Step 2 is of a different kind, and it is the calculus itself. Two working notes treat the syntax of rho as a type constructor: *Where Substitution Happens* parameterises it by a set D of data carried on names and messages and a set V of variables [22], and *Oriented Processes* specialises D to a group G and lets G act on processes [20]. This section expresses that construction in the extensions proposed here. It is a monad, and the group action is a distributive law.

10.1 The construction

In the paired-message calculus of [22], messages are pairs $\langle P, d\rangle$ of a code and a datum $d \in D$, names are quoted messages, and variables form their own syntactic category, occurring directly only under the unary drop $*v$. Communication substitutes a received name for a bound variable, and substitution is *semantic*: it commutes with every constructor, quotation included, and its one non-trivial case is

$$(*v)\{@\langle p, d\rangle/v\} \;=\; p.$$

Separating variables from names is what makes V a parameter: the binding discipline lives in one place and can be chosen independently of the name theory [22].

Oriented Processes works with the pattern-matching variant, in which a receipt binds both the code and the datum of what it receives, and fixes $D = G$. Variables occur *decorated*, $[g]X$ and $[g]\delta$, carrying the action pending on them until they are instantiated. The action $g \cdot -$ multiplies every literal datum on the left at every depth of quotation and is recorded, not applied, at variables: $g \cdot [k]X = [gk]X$. Substitution discharges the record, $([g]X)\sigma = g \cdot \sigma(X)$. On closed terms the action is the image of left multiplication L_g under the functor that the parametric syntax defines [20].

10.2 The syntax monad

Write $\mathsf{Rho}_D(V)$ for the processes over data D whose free variables lie in V. This is a monad in V for each fixed D, and it is the standard monad of syntax with variables and binding: the term is a tree whose leaves are variables, and the monad's bind is substitution at the leaves [6, 4, 7].

- $\mathsf{unit}(v) = *v$: a variable, as the process that runs it.
- $\mathsf{flatMap}(P, \sigma)$, for $\sigma : V \to \mathsf{Rho}_D(W)$, replaces every $*v$ in P by $\sigma(v)$, under quotation and avoiding capture. A receipt's body lives in $\mathsf{Rho}_D(V + 1)$, and bind extends σ by the identity on the bound variable as it passes the binder.

The left unit law is the base case of the definition, the right unit law says that substituting each variable for itself is the identity, and associativity is the substitution lemma. All three are properties of capture-avoiding substitution, not of any particular theory.

That last observation shapes the surface. For a syntax monad, the `Monads` builder does not need the interpretation written out by cases. The toolchain already generates capture-avoiding substitution from the binders declared in `Terms` [15], so it suffices to say which constructor is the variable former:

```
Theory ParamRho() {
  Types {
    Proc(D, V); Name(D, V); Msg(D, V);
  }
  Terms {
    PZero . |- "0" : Proc(D,V);
```

```
    PDrop . v:V |- "*" v : Proc(D,V);
    POut . x:Name(D,V), m:Msg(D,V) |- x "!" "(" m ")" : Proc(D,V);
    PIn . x:Name(D,V), ^v.p:[V -> Proc(D,V)]
          |- "for" "(" v "<-" x ")" p : Proc(D,V);
    PPar . ps:HashBag(Proc(D,V)) |- "{" ps.*sep("|") "}" : Proc(D,V);
    PBDrop . x:Name(D,V), u:Name(D,V) |- "*" "(" x "," u ")" : Proc(D,V);
    MPair . p:Proc(D,V), d:D |- "<" p "," d ">" : Msg(D,V);
    NQuote . m:Msg(D,V) |- "@" m : Name(D,V);
  }
  Equations {
    (PBDrop (NQuote m) u) == (POut u m); // E2: the binary drop
  }
  Rewrites {
    RComm : (PPar {(PIn x ^v.p), (POut x m), ...rest})
            ~> (PPar {(subst ^v.p (MCode m)), ...rest});
    RPar : if S ~> T then (PPar {S, ...rest}) ~> (PPar {T, ...rest});
  }
  Monads {
    Rho over Proc(D, _) by substitution at PDrop;
    toBag : Occurrences; ordered : Positions;
    laws : generated;
  }
}
```

Three features of the declaration are new, and each is a small, definite addition to the proposal.

`over Proc(D, _).` The monad is in one parameter; the other is fixed. This is the general shape of a parametric type constructor in the DDL, and the hole marks the monadic position.

`by substitution at PDrop.` A syntax monad is declared by naming its variable former. flatMap is the generated substitution, specialised so that the variable former is the leaf: a substitution $\sigma : V \to \mathsf{Rho}_D(W)$ replaces each `(PDrop v)` by $\sigma(v)$. This is exactly semantic substitution, because variables occur directly only under the drop.

`laws : generated.` The laws of a syntax monad follow from the generated substitution's own correctness, so they are discharged once, for the toolchain, and not once per theory. This is a better position than the prelude collections are in, whose laws are discharged monad by monad.

Remark 10.1 (The communication rule is a bind). In `RComm` the reduct `(subst ^v.p (MCode m))` is $\mathsf{flatMap}(p, \sigma)$ for the Kleisli arrow that sends the bound variable to the code of the received message — `MCode` is the projection taking `(MPair p d)` to `p` — and every other variable to itself. So every communication performs one bind in the syntax monad: the space monad of §1 delivers its datum by binding the continuation in the data monad of the calculus's own syntax. The two comprehensions meet exactly at the witness. The envelope erasure of [22] — a receiver can never record the datum of the message it receives — has a one-line reading: the Kleisli arrow of `RComm` factors through the projection $\mathsf{Msg}_D(V) \to \mathsf{Rho}_D(V)$ that forgets D. The pattern-matching binder of [20], which binds code and datum together, replaces that arrow with one into a two-sorted substitution that forgets nothing.

Remark 10.2 (Cost). The per-binding charge of §7 applies to a user-level `foreach` over the syntax monad. It is not proposed for the bind inside `RComm`, which remains part of a single witnessed communication and is charged as one, as it is today. Charging that substitution per occurrence would change the cost model of the core, which this note does not do.

10.3 The functor in D

D occurs only covariantly, in the datum of a message, so $\mathsf{Rho}_{(-)}(V)$ is a functor in D for each V: a map $f : D \to D'$ acts on every datum, at every depth of quotation. Step 2 should generate map for every covariant sort parameter, in the manner of a derived functor instance; no declaration is needed. With $D = G$, the action of [20] on closed terms is this generated map at left multiplication:

$$g \cdot P \;=\; \mathsf{map}_D(L_g)(P) \qquad (P \text{ closed}).$$

On open terms the generated map is not enough. It moves the literal data but leaves variables alone, and a variable that is later instantiated would then escape the action. That is the problem deferral solves, and its solution is a second monad.

10.4 Orientation is a distributive law

Let $\mathsf{Frame}_G(X) = G \times X$, the writer monad of the group G viewed as a monoid. For a group it is the free G-set monad: $\mathsf{unit}(x) = (e, x)$, and binding multiplies frames. Its elements are exactly the decorated variables of [20]: $[g]X$ is the pair (g, X), and X is $[e]X$. Data are decorated the same way, with a literal datum g read as g applied to the origin, so the data of the oriented calculus are $\mathsf{Frame}_G(\Delta + 1)$ for data variables Δ.

The oriented calculus is the composite

$$\mathsf{Orient}_G \;=\; \mathsf{Rho}_{\mathsf{Frame}_G(\Delta+1)} \circ \mathsf{Frame}_G,$$

processes whose variables carry frames, and the action is the distributive law that makes the composite a monad [5]:

$$\lambda : \mathsf{Frame}_G \circ \mathsf{Rho} \Rightarrow \mathsf{Rho} \circ \mathsf{Frame}_G, \qquad \lambda(g, P) = g \cdot P.$$

It takes a frame sitting outside a process and pushes it to the leaves: multiplying data by the generated map, and recording at each variable. The composite's bind is substitution that discharges the record, $([g]X)\sigma = g \cdot \sigma(X)$, which is the substitution of [20] verbatim.

Proposition 10.3 (The action is a distributive law)**.** *λ satisfies the four laws of a distributive law of* Frame_G *over* Rho*, and each is a result already proved in [20].*

Law	In the oriented calculus	Where it is proved
unit of Rho	$g \cdot [e]X = [g]X$	the definition of the action, at a variable
unit of Frame_G	$e \cdot P = P$	the automorphism lemma, part (i)
multiplication of Frame_G	$g \cdot (h \cdot P) = (gh) \cdot P$	the automorphism lemma, part (i)
multiplication of Rho	$g \cdot (P\sigma) = (g \cdot P)\sigma$	the lemma on substitution and matching

Sketch. The first two laws are the clauses of the definition and the identity case of the automorphism lemma. The third is its composition case. For the fourth, both sides agree at a decorated variable, where $(g \cdot [k]X)\sigma = (gk) \cdot \sigma(X) = g \cdot (([k]X)\sigma)$, and at every other constructor by induction; this is the calculation of the substitution lemma, and deferral is what makes σ the same on both sides. □

The proposition also says why deferral was the right design in [20]. Without it, the action would be the generated map in D alone, which commutes with nothing at the variables, and the composite would not be a monad. With it, Orient_G is a monad, and everything the extensions of this note offer a monad becomes available to oriented processes.

In the proposed surface the construction is one theory, parameterised by a group:

```
Theory OrientedRho(grp: u.Group) {
  ParamRho() \/ grp
    Monads {
      Frame over Writer(Elem); // prelude: X |-> Elem x X
      Orient = Rho after Frame by generated action;
      toBag : Occurrences; ordered : Positions;
      laws : generated;
    }
}
```

`Writer` is a prelude monad parameterised by a monoid, here the group's carrier; it is the first instance of a monad transformer as a theory parameter (§3.3). `after ... by` composes two declared monads through a distributive law, and `generated action` names the law the toolchain can build: the generated map in D for data, and the record at the variable former for variables. Its four obligations are the rows of the table above, and for `OrientedRho` they are already discharged. The chaos constant, idempotent summation and the oriented communication rule of [20] are added as further `Terms`, `Equations` and `Rewrites` in the ordinary way; none of them touches the monad, because none of them is a variable former.

10.5 The capabilities

Entry	Rho and Orient_G	**Consequence for `foreach`**
`toBag`	the multiset of variable occurrences; a monad morphism, since the occurrences of $\mathsf{flatMap}(P, \sigma)$ are the union, over the occurrences v of P, of the occurrences of $\sigma(v)$	`foreach` visits occurrences; the measure counts them, on canonical forms (below)
`ordered`	yes: positions in the canonical form enumerate the occurrences	ordered aggregators are allowed (Stage 3)
`commutative`	no; syntax monads are not commutative, and the commutativity of `|` does not make them so	`&` is free under commutative aggregators and refused under ordered ones

The measure needs canonical forms. Obligation 3.3 requires toBag to be well defined on the theory's quotient: congruent terms must have the same bag. For Rho the count of variable occurrences is invariant under structural congruence: the monoid laws and E2 neither duplicate nor erase variables. For Orient_G it is not. Distributivity, $P \mid (Q + R) \equiv P \mid Q + P \mid R$, duplicates the occurrences of P, and chaos, $\mathbf{1} \mid P \equiv \mathbf{1}$, erases them. Two congruent terms then have different bags, and since matching is modulo the equations, a receiver cannot choose which representative it is sent. toBag must be taken on a canonical form, which the node already computes as the section of the structural quotient used for hashing [27], and for Orient_G that form is the distributed one: the measure, and so the precharge, is the size of the expanded term, which can be exponentially larger than the source.

10.6 `foreach` over a process

At run time, rho reduces closed processes, and a quoted process received on a channel has no free variables. A `foreach` over its object variables would have measure zero. The useful instance is the one in which V is a data sort of *holes*: $\mathsf{Rho}_D(\mathsf{HoleId})$ is the sort of process templates, closed as far as the calculus is concerned, whose leaves are holes to be filled, in the manner of the typed holes of the plausible-fiction marketplace [21].

Filling a template is the monad's bind, and the bind is not a commutative monoid on contributions, so it is not an aggregator. It is reached in two steps that keep the comprehension pure. The aggregator collects a *substitution* — finite maps from hole occurrences to processes, under union, a commutative monoid because each occurrence is visited once and every union is therefore disjoint — and the consumer applies it:

```
for( @tmpl <- templates where tmpl.holes() <= 32 ) {
  {% foreach( h <- tmpl where Fillable(h) ) yield( fills ) by Subst()
      {% { h : Fill(h) } %}[t] %}[s]
| for( @sub <- fills ){ filled!( tmpl.subst( sub ) ) }
}
```

Each surviving occurrence contributes a singleton map whose value is the lookahead of `Fill(h)`; a rejected occurrence contributes the empty map, and its hole stays a hole. The outer gate draws one token per hole occurrence, the inner one per filled hole, and the guard on the receipt is the bound the precharge proof reads (§7.3). Under the default aggregator the same comprehension would publish `Fill(h1) | Fill(h2) | ...`: the fills side by side and detached from the template, which is meaningful, and different. Over Orient_G each hole carries a frame, and `Fill(h)` is written in the hole's own coordinates and transported when the consumer applies the bind: the local frames of [20], delivered through the comprehension.

10.7 Symmetry reduces the precharge

The orbit-counting theorem of [20] has a direct consequence for cost.

Proposition 10.4 (Symmetry-reduced precharge)**.** *Let P be an oriented template with finitely many hole occurrences, let $H = \mathrm{Stab}(P)$, and suppose the body and guard of a `foreach` over P are H-equivariant: $k([h]x) = h \cdot k(x)$ and the guard holds at $[h]x$ exactly when it holds at x. Then the comprehension needs only one body lookahead per H-orbit of hole occurrences, the number of orbits is $\mathsf{measure}(P)/|H|$, and that number is an integer.*

Sketch. Hole occurrences are atoms in the sense of [20], and the action maps them to hole occurrences, so they form an H-invariant subset of the free atoms. The stabiliser acts freely on the free atoms by the orbit-counting theorem, hence freely on this subset, so every orbit has exactly $|H|$ members and $|H|$ divides their number. By equivariance, the normal form of the body at $[h]x$ is the action of h on the normal form at x, which is applied as a recorded frame rather than computed by a fresh lookahead. □

When the hypotheses hold, the inner lane's precharge is divided by $|H|$, and the divisibility theorem is what makes the reduced figure exact rather than a rounded estimate. The outer lane is unchanged: every occurrence is still a binding and still contributes, so the reduction applies to the cost of looking ahead at bodies and guards, not to the traversal or the accumulation. This is the symmetry reduction of [20] applied to a comprehension instead of to a state space: the symmetry is read off the template, not declared.

10.8 Dynamic frames, and where the word problem lands

The final extension of [20] lets decorations be words in received data, $[w]X$ with w in the free product of G with the free group on the data variables. In the terms of this note, the frame monad changes from the writer $G \times (-)$ to the free-group-over-G monad $\Delta \mapsto G * F(\Delta)$, whose bind evaluates words by substituting words for variables. The composite is formed the same way.

What changes is equality. Matching in f1r3|@ng is modulo the equations of the theory, and the frame sort is now the carrier of `u.Group` with variables. Deciding that two decorations are equal is matching modulo the group axioms in the `Equations` block of `Group`, which is to say the word problem for $G * F(\Delta)$. The cost that [20] identifies is therefore not a new mechanism. It is the second rung of the DDL — matching up to equations — meeting a theory whose equational problem is as hard as the group's word problem, and it is paid wherever such terms are matched, including in the guards of a `foreach`.

11 A first tranche

1. The prelude declares `Seq`, `Bag` and the finite powerset on the three built-in collections, each with `toBag`, with `ordered` on `Seq` and `commutative` on the other two.
2. `foreach` with `<-`, `;` between groups, `&` within them, crisp `where`, and `yield( rslt )`.
3. Lookahead for guards and bodies, with the terminating fragment $\mathcal{T}$ given by funding: a body is admitted when it is wrapped in a finite inner budget.
4. Stage 1 aggregation: `by` a theory instance descending from `CommutativeMonoid`, checked by origin, defaulting to `ParMonoid`; the diagnostics `[aggregator-not-commutative]` and `[aggregator-ambiguous]`.
5. The cost-accounted surface with both gates, per-binding outer charge, and the n-ary outer gate for single-group comprehensions.
6. `foreach` on the left of `;`.
7. Native evaluation in the reducer, the new process node gated on protocol version, and the demand rule in the funding analysis.
8. The laws, the bag morphism, confluence, the fragment obligation and the aggregator obligations recorded, and discharged for the prelude monads and for `ParMonoid`.

Deferred: sort parameters and user-declared monads (Step 2), with the parametric rho calculus of §10 as their acceptance example; syntax monads declared by substitution; composition through distributive laws; aggregation Stages 2–4; graded `where` in `foreach`; the lax modality.

12 Open questions

Open question 12.1 (The terminating fragment). Is funding the right definition of $\mathcal{T}$ for the first tranche, or should a cheaper syntactic fragment be admitted alongside it? And does the recorded uniqueness of normal forms on the funded fragment cover the bodies developers will write, or must $\mathcal{T}$ exclude internal races explicitly?

Open question 12.2 (Nondeterministic bodies). Should a body outside the deterministic fragment be admitted with its result declared nondeterministic, or refused? Admitting it keeps `foreach` total over $\mathcal{T}$'s terminating part; refusing it keeps every published result a function of the source.

Open question 12.3 (The inner lane). When a body's lookahead exhausts its budget, nothing is published and the outer tokens are spent. Should the inner budget for every candidate be reserved

atomically at entry, alongside the outer gate, or is the burn acceptable given that the funding proof must cover the inner lane anyway?

Open question 12.4 (Dependent groups). For a `foreach` with dependent groups, should the outer gates be taken group by group as the deficit of §4.7 unfolds, accepting that a later group may find its lane empty after earlier groups have paid, or should a declared upper bound be taken at entry, accepting overpayment when the bound is loose?

Open question 12.5 (Per-element funding). May the inner signature mention variables bound by the pattern, `{% P %}[#x]`? It lets data pay for its own processing, at the price of data-dependent lanes in the funding proof.

Open question 12.6 (Semilattice in the tower). Stage 2 needs a `Semilattice` theory, and Stage 4 a `Semimodule` over a rig. Should both join the universal-algebra module now, so that the origin checks of later stages have something to descend from?

Open question 12.7 (Arity drift). Whatever is decided for arity-changing replacements of `PPar` must be decided for monad interpretations and for aggregator operations. Which resolution is preferred?

Open question 12.8 (Syntax monads). Should every theory with a variable former get its syntax monad, declared `by substitution at` that former, with laws discharged once for the toolchain, and with toBag always taken on the node's canonical form?

Open question 12.9 (Generated distributive laws). For which group actions on a theory can the toolchain generate the distributive law, as it can for the deferred action of oriented processes, and how should a law that cannot be generated be supplied and its four obligations stated?

Open question 12.10 (Default monad). Should a sort with exactly one declared monad allow the qualifier to be omitted, as proposed, or should the qualifier always be required for readability?

References

[1] P. N. Benton, G. M. Bierman, and V. C. V. de Paiva. Computational types from a logical perspective. *Journal of Functional Programming*, 8(2):177–193, 1998.

[2] J. Dean and S. Ghemawat. MapReduce: simplified data processing on large clusters. In *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 137–150, 2004.

[3] M. Fairtlough and M. Mendler. Propositional lax logic. *Information and Computation*, 137(1):1–33, 1997.

[4] T. Altenkirch and B. Reus. Monadic presentations of lambda terms using generalized inductive types. In *Computer Science Logic (CSL)*, LNCS 1683, pages 453–468, 1999.

[5] J. Beck. Distributive laws. In *Seminar on Triples and Categorical Homology Theory*, Lecture Notes in Mathematics 80, pages 119–140. Springer, 1969.

[6] R. S. Bird and R. Paterson. De Bruijn notation as a nested datatype. *Journal of Functional Programming*, 9(1):77–91, 1999.

[7] M. Fiore, G. Plotkin, and D. Turi. Abstract syntax and variable binding. In *Proceedings of the 14th IEEE Symposium on Logic in Computer Science (LICS)*, pages 193–202, 1999.

[8] A. Kock. Monads on symmetric monoidal closed categories. *Archiv der Mathematik*, 21:1–10, 1970.

[9] E. Meijer, B. Beckman, and G. Bierman. LINQ: reconciling object, relations and XML in the .NET framework. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, page 706, 2006.

[10] E. Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991.

[11] P. Wadler. List comprehensions. In S. L. Peyton Jones, *The Implementation of Functional Programming Languages*, chapter 7. Prentice Hall, 1987.

[12] P. Wadler. Comprehending monads. *Mathematical Structures in Computer Science*, 2(4):461–493, 1992.

F1R3FLY.io working notes. Each lives in the F1R3FLY.io `publications` repository, in the directory given.

[13] L. G. Meredith. Cost-accounted rho calculus: a spectral decomposition of phlogiston. F1R3FLY.io working note, 2026. `cost-accounting/`.

[14] L. G. Meredith. Continued interactive GSLTs and the cost endofunctor. F1R3FLY.io working note, 2026. `cost-accounting-as-monad/`.

[15] L. G. Meredith. f1r3|@ng for the working software developer. F1R3FLY.io working note, 2026. `MeTTaIL4WorkingDev/f1r3lang4dev/`.

[16] L. G. Meredith. Graded where-clauses: non-crisp truth values as the syntax of non-determinism resolution in the rho calculus. F1R3FLY.io working note, 2026. `fuzzyware/`.

[17] L. G. Meredith. Graph-structured lambda theories: a reference for the working developer. F1R3FLY.io working note, 2026. `GSLT-intro/`.

[18] L. G. Meredith. History as an endofunctor: reversibility, covering spaces, and the memory price of energy conservation in cost-accounted GSLTs. F1R3FLY.io working note, 2026. `history-functor/`.

[19] L. G. Meredith. Two erasures and two laxities: internalising the cost and history monads over continued interactive GSLTs. F1R3FLY.io working note, 2026. `history-and-cost/`.

[20] L. G. Meredith. Oriented processes: group-valued data, a chaos constant, and an equivariant COMM for the reflective higher-order calculus. F1R3FLY.io working note, draft 3, 2026. `process-rotations/`.

[21] L. G. Meredith. Plausible fiction on f1r3|@ng: a requirements and design document for a typed marketplace of programmatic intentions. F1R3FLY.io working note, 2026. `plausible-fiction/`.

[22] L. G. Meredith. Where substitution happens: semantic substitution, deferred unquote and explicit substitution in a paired-message rho calculus. F1R3FLY.io working note, 2026. `OnSemanticSubstitution/`.

[23] M. Stay, L. G. Meredith, and J. B. Wells. Generating hypercubes of type systems. F1R3FLY.io working note, 2026. `drafts/Hypercube/`.

[24] L. G. Meredith. Weighted graph-structured lambda theories: stochastic and quantum execution for f1r3|@ng. F1R3FLY.io working note, second version, 2026. `rho-life/`.

[25] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 141(5):49–67, 2005.

Software.

[26] F1R3FLY.io. f1r3node-rust, branch feature/module-syntax: the mettail-elab reference elaborator for the Module and Theory surface. https://github.com/F1R3FLY-io/f1r3node-rust.

[27] F1R3FLY.io. f1r3node-rust, branch feature/f1r3lang-mettail-only: the frontend admission contract (docs/design/f1r3lang-frontend.md), the evaluation composition (docs/node/f1r3lang-evaluation.md), the cost-monad correspondence record and graded adequacy development (docs/theory/), the synchronous-send normaliser, and the funding analysis (rholang/src/rust/interpreter/accounting/delta_sigma.rs). https://github.com/F1R3FLY-io/f1r3node-rust.